ChavScript

Do Meme ao Código — uma linguagem de programação inspirada em **Chaves**

Projeto de Compiladores · Tradutor para JavaScript · Cena real: "O Dinheiro Perdido" (1978)

Análise léxicaAnálise sintáticaAnálise semânticaGeração de código

1. Contextualização e objetivo

Linguagens de programação não precisam de palavras-chave tradicionais (`if`, `while`, `print`) — podem nascer de memes, personagens e cultura da internet, como a **BIRL** (Kleber Bambam).

Objetivo do trabalho

- Criar uma **linguagem própria** inspirada num meme reconhecível
- Implementar um **tradutor** com as etapas de compiladores: análise léxica, sintática, semântica e geração de código
- Converter programas ChavScript para **JavaScript/Node.js** executável
- Identificar e informar **erros** com número da linha

2. O meme escolhido: Chaves

- **Chaves (El Chavo del Ocho)**, de Roberto Gómez Bolaños (Chespirito) — um dos personagens mais icônicos da cultura pop brasileira
- Frases e situações viraram memes: "o senhor tem alguma coisa contra a panela?", o aluguel do Seu Madruga, "sem querer querendo"
- A série já tinha natureza **pedagógica** (a Vila ensinava as crianças) — a linguagem mantém esse espírito

Correspondência meme → programação

Expressão da Vila	Comando ChavScript
Personagens conversando	`CONVERSA ENTRE CHAVES E QUICO ... CENA TERMINA`
Falas e ações	`DIZ / RESPONDE / PENSA / GRITA / ENTRA / SAI`
Decisão na trama	`SE ... IGUAL/DIFERENTE/MAIOR/MENOR ... SENÃO ... FIMSE`
Cena se repete	`REPETE n VEZES ... FIM`

Pergunta ao público	`PERGUNTA expr EM var` (entrada de dados)
Variáveis	`TEXTO nome = "..."` / `NUMERO n = 2`

3. Arquitetura do tradutor

Código .chav→ LEXER

tokens + erros léxicos→ PARSER

AST (gramática EBNF)→ SEMÂNTICA

tipos + símbolos→ JS / Node

código executável

Detalhes

- **Lexer** próprio: reconhece palavras reservadas, identificadores, números, textos, operadores `+ - * / ( )` e comentários `//`; reporta símbolo inválido com linha
- **Parser** descendente recursivo com precedência de expressões; gramática formal em EBNF (aba Gramática do estúdio)

- **Semântica**: tabela de símbolos, variáveis não declaradas/redeclaradas, tipos incompatíveis em contas e comparações, personagem fora do elenco
- **Interpretador** assíncrono gera os eventos da cena animada (estúdio com personagens SVG, cenários e voz)
- **Codegen**: emite JavaScript/Node.js válido — variáveis viram `let`, `SE` vira `if/else`, `REPETE` vira `for`, `PERGUNTA` lê stdin

4

4. Exemplo: ChavScript → JavaScript

Programa que recria a cena real **"O Dinheiro Perdido"** (1978) com os 5 personagens do episódio:

```
VILA COMEÇA

CONVERSA ENTRE CHAVES E CHIQUINHA E QUICO E SEU_MADRUGA E SR_BARRIGA

  NUMERO socos = 4

  TEXTO lugar = ""

  ... (5 personagens entram em cena)

  CHAVES PERGUNTA "Onde o senhor viu o dinheiro pela última vez?" EM lugar
```

```
  SE lugar IGUAL "bolso"
    SEU_MADRUGA GRITA "O que te interessa?! Mas o dinheiro é meu, hein?"
    REPETE socos VEZES
      SEU_MADRUGA DIZ "Direita! Esquerda! Outra direita!"
      CHAVES RESPONDE "Ai! Ai! Ai!"
    FIM
  SENÃO
    CHAVES DIZ "Não encontrei dinheiro nenhum, juro!"
  FIMSE
CENA TERMINA
VILA TERMINA
```

Gera JavaScript equivalente e **executável** (`node programa.js < respostas.txt`):

```
let SOCOS = 4; let LUGAR = "";

LUGAR = __pergunta(""+"Onde o senhor viu o dinheiro pela última vez?");

if(LUGAR === "bolso"){

  for(let __i=0;__i<(SOCOS);__i++){

    console.log("Seu Madruga: "+"Direita! Esquerda! Outra direita!");

    console.log("Chaves: "+"Ai! Ai! Ai!");

  }

}
```

5

5. Requisitos mínimos — cobertura

Requisito	Implementação ChavScript
Declaração de variáveis	`TEXTO` / `NUMERO` nome = valor

Tipos de dados	texto e número, verificados estaticamente
Aritmética	`+ - * /`, parênteses e precedência
Entrada de dados	`PERGUNTA expr EM var` — input no estúdio, stdin no Node
Saída de dados	`DIZ / RESPONDE / PENSA / GRITA`
Condicional if/else	`SE ... SENÃO ... FIMSE` com 4 operadores relacionais
Repetição	`REPETE n VEZES ... FIM` (aninhável)
Comentários	`//` até o fim da linha
Tratamento de erros	léxicos, sintáticos e semânticos com linha
Geração de código	JavaScript/Node.js executável

Extras: estúdio animado (personagens, cenários, voz), 6 personagens, conversas com N participantes, diálogo automático por assunto, histórico de execuções.

6. Tratamento de erros (demonstração)

O compilador detecta erros nas três fases, sempre com o número da linha:

```
CHAVES DIZ "Oi" @        →  Linha 5: símbolo inválido @        (léxico)

SE n MAIOR 0             →  Linha 8: bloco SE sem fechamento    (sintático)

  CHAVES DIZ "Oi"

NUMERO x = nome - 2      →  Linha 6: operador - só vale p/ NUMERO  (semântico)

CHAVES DIZ sobrenome     →  Linha 7: variável não declarada
```

4 programas de teste válidos + 4 exemplos de erro em `testes/`, verificados por `node testes/testar.js` → **TODOS OS TESTES PASSARAM**.

7. Demonstração prática (roteiro)

1. **Vídeo real primeiro:** episódio "O Dinheiro Perdido" no YouTube (Parte 1: youtube.com/watch?v=pzcAyxehVVQ · Parte

2: youtube.com/watch?v=UUowj7zxsMg)

2. **Estúdio:** abrir `ChavScript.html` → exemplo já carregado → **Compilar** (mostrar fases no console) → **Executar** → responder `bolso` na pergunta → 5 personagens em cena
3. **Erro:** trocar um comando por algo inválido e mostrar a mensagem com linha
4. **Codegen: Gerar JavaScript** → aba JS → baixar `.js` → rodar `node programa.js < respostas.txt`
5. **Testes:** rodar `node testes/testar.js` no terminal

8. Entregáveis

Entregável	Onde está
Documento de especificação (PDF)	`ESPECIFICACAO.md` → `especificacao.pdf`
Código-fonte do tradutor	`ChavScript.html` — projeto completo em um arquivo
Programas de teste	`testes/` — 4 válidos (1 com entrada+decisão+repetição) + 4 de erro
Evidências de funcionamento	`testes/saida/*.js` — código gerado e saída executada

Apresentação	este documento + demonstração ao vivo do estúdio

Referência do trabalho: BIRL — Bambam's "It's show time" Recursive Language. ChavScript segue a mesma ideia (expressões populares viram comandos) com sintaxe e implementação próprias.

9. Conclusão

- O ChavScript mostra que uma linguagem pode nascer de um meme e ainda assim ter **estrutura formal**: gramática, tokens, tipos e fases de compilação
- O tradutor **não é substituição de texto**: cada fase produz uma representação intermediária (tokens → AST → eventos → JavaScript)
- O resultado é uma linguagem que **realmente funciona**: compila, executa no estúdio animado e gera código Node.js independente

Obrigado! — perguntas?